

```

import os
from datetime import datetime, timedelta, timezone
from typing import Optional

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from passlib.context import CryptContext
from sqlalchemy.orm import Session

import models
from database import get_db
from schemas import TokenData

# --- Config ---
# In production, load SECRET_KEY from an environment variable / secrets manager.
SECRET_KEY = os.getenv("JWT_SECRET_KEY", "dev-secret-change-me-in-production")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (
        expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
) -> models.User:

```

```

credentials_exception = HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Could not validate credentials",
    headers={"WWW-Authenticate": "Bearer"},
)
try:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    username: str = payload.get("sub")
    if username is None:
        raise credentials_exception
    token_data = TokenData(username=username)
except JWTError:
    raise credentials_exception

user = db.query(models.User).filter(models.User.username == token_data.username)
if user is None:
    raise credentials_exception
return user

```